Thala Staked LPT

Security Assessment

April 11th, 2025 — Prepared by OtterSec

Andreas Mantzoutas andreas@osec.io

Robert Chen r@osec.io

Table of Contents

Executive Summary **2**

Overview 2

Key Findings 2

Scope **3**

Findings **4**

Vulnerabilities **5**

OS-TSL-ADV-00 | Uncapped XLPT Minting 6

General Findings **7**

OS-TSL-SUG-00 | Blocked Claiming on Zero Supply 8

OS-TSL-SUG-01 | Code Maturity 9

Appendices

Vulnerability Rating Scale **10**

Procedure **11**

© 2025 Otter Audits LLC. All Rights Reserved.

01 — Executive Summary

Overview

Thala Protocol engaged OtterSec to assess the `staked-lpt-oracle` and `thala-staked-lpt` programs. This assessment was conducted between April 6th and April 10th, 2025. For more information on our auditing methodology, refer to Appendix B.

Key Findings

We produced 3 findings throughout this audit engagement.

In particular, we identified a vulnerability concerning the lack of a rate limit, which may allow users to mint unlimited `xLPT` tokens without proper backing, enabling them to exploit the system by utilizing unbacked `xLPT` as collateral (OS-TSL-ADV-00).

We also made suggestions to improve functionality and ensure adherence to coding best practices. (OS-TSL-SUG-01). Moreover, we highlighted a possibility where users will not be able to claim rewards because the boost multiplier calculation aborts during farming updates if the total `xLPT` supply drops to zero (OS-TSL-SUG-00).

© 2025 Otter Audits LLC. All Rights Reserved.

02 — Scope

The source code was delivered to us in a Git repository at https://github.com/ThalaLabs/thala-modules. This audit was performed against commit 3579943.

Brief descriptions of the programs are as follows:

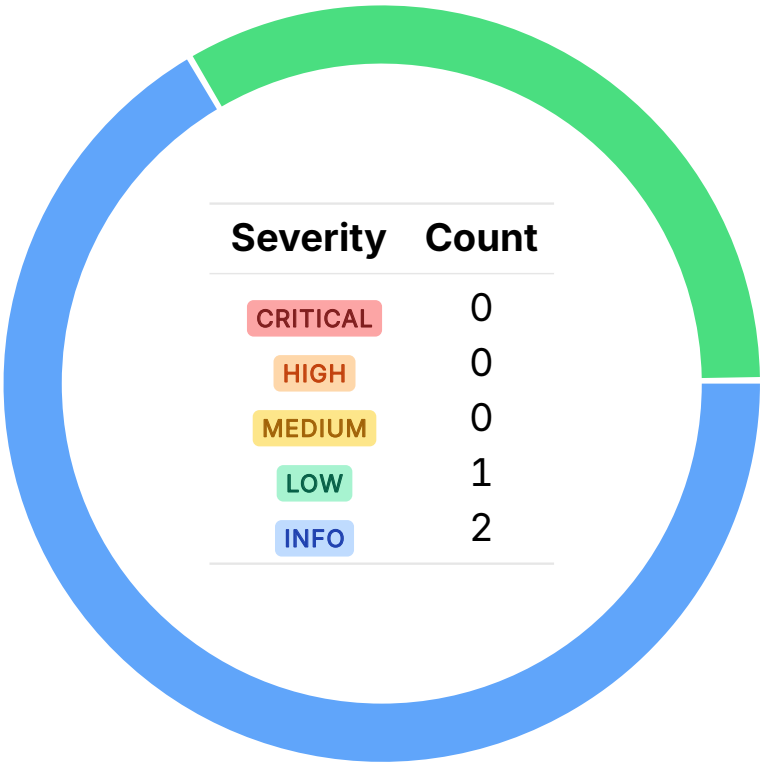
Name	Description
staked-lpt-oracle	An oracle implementation for pricing LPT positions based on the Chainlink stable pool pricing formula. It extends this model by incorporating upscaling and normalization of underlying asset balances for metastable pools.
thala-staked-lpt	Implements a staking system for liquidity provider tokens (LPTs), allowing users to stake LPTs in exchange for “xLPT” tokens that represent their staked positions and entitle them to farming rewards.

© 2025 Otter Audits LLC. All Rights Reserved.

03 — Findings

Overall, we reported 3 findings.

We split the findings into **vulnerabilities** and **general findings**. Vulnerabilities have an immediate impact and should be remediated as soon as possible. General findings do not have an immediate impact but will aid in mitigating future vulnerabilities.



Severity	Count
CRITICAL	0
HIGH	0
MEDIUM	0
LOW	1
INFO	2

© 2025 Otter Audits LLC. All Rights Reserved.

04 — Vulnerabilities

Here, we present a technical analysis of the vulnerabilities we identified during our audit. These vulnerabilities have *immediate* security implications, and we recommend remediation as soon as possible.

Rating criteria can be found in Appendix A.

ID	Severity	Status	Description
OS-TSL-ADV-00	LOW	RESOLVED	A lack of rate limit may allow users to mint unlimited `xLPT` tokens without proper backing, enabling them to exploit the system by utilizing unbacked `xLPT` as collateral.

© 2025 Otter Audits LLC. All Rights Reserved.

Uncapped XLPT Minting LOW OS-TSL-ADV-00

Description

There is a potential rate-limiting vulnerability in the staking logic related to uncapped `xLPT` minting. If the `stake` function permits minting without appropriate constraints, a user may mint an unlimited amount of unbacked `xLPT`, allowing them to exploit the system by utilizing it as collateral or for farming rewards without sufficient backing.

>_ *lending_core/sources/farming.move* RUST

```rust
fun accrue_user_pool_reward(pool: &Pool, user_pool: &mut UserPool, reward_name: String) {
    let pool_reward = simple_map::borrow(&pool.rewards, &reward_name);
    let pool_acc_reward_per_share = pool_reward.acc_rewards_per_share;
    if (!simple_map::contains_key(&user_pool.rewards, &reward_name)) {
        simple_map::upsert(&mut user_pool.rewards, reward_name, UserPoolReward {
            reward_name,
            last_acc_rewards_per_share: pool_acc_reward_per_share,
            reward_amount: 0,
        });
    };
    [...]
}
```

Remediation

Ensure the minting process is rate-limited.

Patch

Fixed in e429b1d.

© 2025 Otter Audits LLC. All Rights Reserved.

05 — General Findings

Here, we present a discussion of general findings during our audit. While these findings do not present an immediate security impact, they represent anti-patterns and may result in security issues in the future.

ID	Description
OS-TSL-SUG-00	If the total `xLPT` supply drops to zero, users will not be able to claim rewards because `boost_multiplier` aborts during farming updates.
OS-TSL-SUG-01	Recommendations to improve functionality and ensure adherence to coding best practices.

© 2025 Otter Audits LLC. All Rights Reserved.

Blocked Claiming on Zero Supply

OS-TSL-SUG-00

Description

If the total supply of `xLPT` reaches zero, users may be temporarily unable to claim farming rewards. This is because `claim_rewards` invokes `update_farming`, which calls `boost_multiplier`. The assertion in `boost_multiplier` aborts if the total `xLPT` supply is zero, causing the entire reward claim process to fail. As a result, users must re-stake a small amount before they can successfully claim rewards and then proceed to unstake.

>_ *thala_staked_lpt/sources/staked_lpt.move* RUST

```rust
[view]
public fun boost_multiplier<T: key>(staked_lpt_metadata: Object<Metadata>, store: Object<T>):
    ↪  u128acquires Farming {
    [...]
    let staked_lpt_total_supply = option::extract(&mut
        ↪  fungible_asset::supply(staked_lpt_metadata));
    assert!(staked_lpt_total_supply > 0, ERR_STAKED_LPT_TOTAL_SUPPLY_ZERO);
    [...]
}
```

Remediation

Update `update_farming` to directly set `new_stake_amount` to 0 if the user's balance is zero, avoiding a `boost_multiplier` call that may fail.

Patch

The developers acknowledged this issue as an accepted risk.

© 2025 Otter Audits LLC. All Rights Reserved.

Code Maturity OS-TSL-SUG-01

Description

1. `staked_lpt::update_rate_limit_whitelist_users` currently borrows the `RateLimitWhitelist` resource multiple times, once in each iteration of the `addresses_to_add` and `addresses_to_remove` loops. This repeated borrowing is inefficient, as the same resource is accessed redundantly. It will be more optimal to borrow the resource once at the beginning of the function and re-utilize the reference.

   >_ thala_staked_lpt/sources/staked_lpt.move RUST

   ```rust
   public entry fun update_rate_limit_whitelist_users(manager: &signer, addresses_to_add:
       ↪ vector<address>, addresses_to_remove: vector<address>) acquires RateLimitWhitelist
       ↪ {
       assert!(manager::is_authorized(manager), ERR_STAKED_LPT_UNAUTHORIZED);
       // Add addresses
       vector::for_each(addresses_to_add, |address| {
           let rate_limit =
               ↪ borrow_global_mut<RateLimitWhitelist>(package::package_address());
           if (!vector::contains(&rate_limit.whitelisted_users, &address)) {
               vector::push_back(&mut rate_limit.whitelisted_users, address);
           }
       });
       [...]
   }
   ```

2. `oracle::get_price_unsafe` returns zero when the LP token price hits circuit breaker limits, even if the price is otherwise valid. This limits flexibility for consumers who may want to apply their own thresholds or still utilize the price. Return the actual computed price along with a `circuit_broken` status, preserving transparency while still signaling risk.

Remediation

Implement the above-mentioned suggestions.

Patch

1. Issue #1 fixed in 5032cfd.
2. Issue #2 fixed in 5032cfd.

© 2025 Otter Audits LLC. All Rights Reserved.

A — Vulnerability Rating Scale

We rated our findings according to the following scale. Vulnerabilities have immediate security implications. Informational findings may be found in the General Findings.

CRITICAL

Vulnerabilities that immediately result in a loss of user funds with minimal preconditions.

Examples:

- Misconfigured authority or access control validation.
- Improperly designed economic incentives leading to loss of funds.

HIGH

Vulnerabilities that may result in a loss of user funds but are potentially difficult to exploit.

Examples:

- Loss of funds requiring specific victim interactions.
- Exploitation involving high capital requirement with respect to payout.

MEDIUM

Vulnerabilities that may result in denial of service scenarios or degraded usability.

Examples:

- Computational limit exhaustion through malicious input.
- Forced exceptions in the normal user flow.

LOW

Low probability vulnerabilities, which are still exploitable but require extenuating circumstances or undue risk.

Examples:

- Oracle manipulation with large capital requirements and multiple transactions.

INFO

Best practices to mitigate future security risks. These are classified as general findings.

Examples:

- Explicit assertion of critical internal invariants.
- Improved input validation.

© 2025 Otter Audits LLC. All Rights Reserved.

B — Procedure

As part of our standard auditing procedure, we split our analysis into two main sections: design and implementation.

When auditing the design of a program, we aim to ensure that the overall economic architecture is sound in the context of an on-chain program. In other words, there is no way to steal funds or deny service, ignoring any chain-specific quirks. This usually requires a deep understanding of the program's internal interactions, potential game theory implications, and general on-chain execution primitives.

One example of a design vulnerability would be an on-chain oracle that could be manipulated by flash loans or large deposits. Such a design would generally be unsound regardless of which chain the oracle is deployed on.

On the other hand, auditing the program's implementation requires a deep understanding of the chain's execution model. While this varies from chain to chain, some common implementation vulnerabilities include reentrancy, account ownership issues, arithmetic overflows, and rounding bugs.

As a general rule of thumb, implementation vulnerabilities tend to be more "checklist" style. In contrast, design vulnerabilities require a strong understanding of the underlying system and the various interactions: both with the user and cross-program.

As we approach any new target, we strive to comprehensively understand the program first. In our audits, we always approach targets with a team of auditors. This allows us to share thoughts and collaborate, picking up on details that others may have missed.

While sometimes the line between design and implementation can be blurry, we hope this gives some insight into our auditing procedure and thought process.

© 2025 Otter Audits LLC. All Rights Reserved.